

BDI Agent Architectures: A Survey

Lavindra de Silva

lpd25@cam.ac.uk

Felipe Meneguzzi

felipe.meneguzzi@pucrs.br

Brian Logan

b.s.logan@uu.nl

Introduction

For the last 30 years, the BDI agent model has formed the basis for much of the research on architectures for agent systems. Such agents are suited for complex, partially observable, and dynamic environments where multiple goals need to be pursued in parallel. Starting with Bratman’s philosophical work, and implementations such as PRS, the theory and practice of BDI agents have proceeded in parallel, with work on semantics leading to new architectural and language features and vice versa.

We survey the most important features of BDI agent architectures and their implementations in agent programming languages.

The BDI Agent Architecture

- **Beliefs** represent the agent’s information about its environment
- **Goals** (desires) are states of affairs to achieve
- **Intentions** are commitments to achieving particular goals
- **Plan-rules** specify when a plan can be used to achieve a goal
- **Plans** typically consist of a sequence of steps
- A **BDI agent** consists of
 - a set of initial beliefs (and in some cases goals)
 - a library of plan-rules

The execution of an agent follows a deliberation cycle

Algorithm 1 BDI Agent Interpreter	
1:	procedure AGENTINTERPRETER($\langle Ev, Bel, PLib, Int \rangle$)
2:	while <i>true</i> do
3:	$Ev \leftarrow \text{UPDATEEVENTS}(Ev)$
4:	$Bel \leftarrow \text{UPDATEBELIEFS}(Ev, Bel)$
5:	$Int \leftarrow \text{SELECTPLANS}(Ev, Bel, PLib, Int)$
6:	$Ev \leftarrow \text{EXECUTEINTENTION}(Int, Ev)$

Approaches to Beliefs

In most agent programming languages, an agent’s belief base is a **conjunction of ground positive literals** in a first-order logical language.

This representation has been enriched in various ways, e.g. **belief formulas** allow negative literals and inference rules (e.g. Horn clauses).

There’s work on extending BDI languages to handle **uncertain beliefs**, and using AI belief revision to ensure **consistency** of a belief base.

Other extensions include: (1) using **description logic** rather than Prolog, allowing for example the sharing of knowledge expressed in ontology languages such as OWL, and (2) **avoiding a commitment** to any particular knowledge representation technology.

Approaches to Goals

Test Goals are goals to test whether a condition holds (is believed by the agent).

Languages take differing approaches to test goals that evaluate to false, e.g. (1) execution of the plan containing the goal blocks until the goal evaluates to true, or (2) a test goal failure triggers a subgoal and the adoption of a plan to make it true.

Achievement goals can be categorised into procedural achievement goals (‘goals to do’) and declarative achievement goals (‘goals to be’).

Some languages that take a procedural view support declarative goals indirectly through **programming idioms**, and in some languages, the specification of an achievement goal may also include a ‘**failure condition**’ specifying the situations in which the goal should be dropped.

Maintenance Goals maintain a particular state of the environment. As with (declarative) achievement goals, maintenance goals give rise to the adoption of plans when the goal condition does not hold, or when a guard condition becomes true.

A **temporally extended goal** is specified by a formula in temporal logic; declarative and maintenance goals can be seen as special cases.

Priorities, deadlines and plan restrictions allow, e.g., utilities and deadlines for goals, and the agent acting to maximise an objective function.

Approaches to Plans

A **plan-rule** typically comprises a **trigger** specifying when the rule is relevant; a **context condition** specifying when the rule is applicable; and a **plan** built from goals, belief update operations, actions, etc.

In the simplest case, the steps in a plan are executed in **sequence**, but many languages give constructs for controlling their order:

- **Series-Parallel Interleaving**: plans are built incrementally by the sequential and parallel compositions of other plans.
- **Arbitrary Interleaving**: allows more fine grained ordering.
- **Non-Interleaving Concurrency**: some languages support ‘true concurrency’, where steps may overlap.
- **Control Flow**: amounts to conditional and looping constructs such as if-then-else statements and while loops.

Approaches to the Deliberation Cycle

In some BDI systems, **failure recovery** is manual (written via ‘failure rules’).

Others support forms of automated failure recovery (e.g. ‘backtracking’).

Intention scheduling is the process of suitably interleaving intentions. Basic approaches involve doing one step in an intention (in each cycle)

- with the highest utility, or in a FIFO / round robin manner
- these can result in conflicts between intentions and/or ‘wasted effort’

Better approaches have been proposed:

- exploiting positive interactions between interleaved intentions, and/or
- avoiding negative interactions between intentions or with the environment

Meta-level reasoning: to choose the most appropriate applicable plan-rule